



Never Stand Still

WK01-02: Crypto Building Blocks

Software Architecture for Blockchain Applications,
COMP6452

Salil Kanhere, Nadeem Ahmed

Agenda

- Learn Crypto building blocks for Blockchains
 - Integrity: Message Digests (Hash)
 - Hash Chains
 - Merkle Trees
 - Secret Keys – Asymmetric/Symmetric
 - Non-Repudiation: Digital Signature

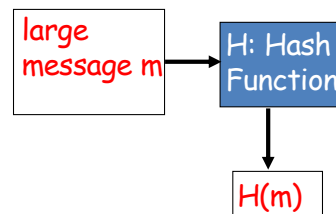
Security Fundamentals

- *confidentiality*: only sender, intended receiver should “understand” message contents
- *authentication*: sender, receiver want to confirm identity of each other
- *message integrity*: sender, receiver want to ensure message not altered (in transit, or afterwards) without detection
- *non-repudiation*: *no one (including the sender) can deny that message was sent by the sender*
- *access and availability*: services must be accessible and available to users

These are the basic security properties we strive to achieve.

Hash/Message Digests (MD)

- Function $H()$ that takes as input an arbitrary length message and outputs a fixed-length string: "message signature"
- The values returned by $H()$ are called hash values, hash codes, digests, or simply hashes.
- $H()$ is often called a "hash function"



- Hashing is the process of converting a large amount of data into a small datum.
- In the figure, given a document, we use a special function to derive a small bit string that sort of becomes a "fingerprint" for the document.
- We chose this function such that it can capture even a minor change in the document by producing a significantly different bit string from the original bit string (see next slide).
- The function can take in arbitrary long input and produce a fix-sized output.
- Such a function is called a hash function, and the resulting bit string is called the hash, hash value, hash code, or digest.
- Some of the popular hash functions are MD5 and variants of SHA like SHA2 and SHA3.
 - MD stands for message digest and SHA stands for Secure Hash Algorithm. SHA-3 belongs to a family of algorithms called **KECCAK**.
 - The hash value produced by these algorithms can be of various lengths like 64-bit, 128, 160, 224, 256 bits and so on.
 - Many BCs use 256 hash values, e.g., Bitcoin use SHA-3-256 and Ethereum uses **KECCAK-256**

Desirable Properties For Hashing

- Easy to calculate
- Deterministic
 - Same message always result in the same hash
- Irreversibility: Can't determine m from $H(m)$
 - One-way
- Collision resistance: Computationally difficult to produce m and m' such that $H(m) = H(m')$
- Seemingly random output
 - Diffusion
 - Change input only one bit, and the hash will change half of the digest bits

- While there are many hash functions, hash functions used for cryptographic purposes usually have the following desirable properties:
 - The hash value should be deterministic where the same input always results in the same hash.
 - A small change to an input/message should change the hash value so extensively that the new hash value appears uncorrelated with the old hash value. This property is useful to prevent someone from guessing a hash value for a modified document.
 - The hash value of a given message or document should be computed fast. There is even specialised hardware to speed up hash calculations.
 - They should be one-way functions.
 - Hence, given a hash, we should not be able to derive the corresponding message. Therefore, the only way to generate a message from its hash value is to try all possible messages.
 - It should be practically infeasible to find 2 different messages with the same hash value. If this happens, it's called a Hash Collision.
 - The possibility of a hash collision can be reduced by increasing the length of the hash value, e.g., most BCs used hash values of 256 bits or more.

Example: SHA-256

- Plaintext

Phishing Explained

Phishing scams are typically fraudulent e-mail messages appearing to come from legitimate sources like your bank, your Internet Service Provider, eBay, or PayPal, for example. These messages usually direct you to a fake web site and ask you for private information (e.g., password, credit card, or other account updates). The perpetrators then use this private information to commit identity theft.

Warning Sign:

URLs Don't Match - Place your mouse over the link in the e-mail message. If the URL displayed in the window of your browser is not exactly the same as the text of the link provided in the message, run. It's probably a fake. Sometimes the URLs do match and the URL is still a fake.

- Hash Digest

```
4720eddc b294a991e712b0e930bd5c61d22ac198219c684ed6d
6cb358ac4d564
```

Take this example where we pass the text to SHA-256 Hash function which produces a specific digest value.

Example: Diffusion in SHA-256

- Plaintext: only change the first letter from P to Q

Phishing Explained

Phishing scams are typically fraudulent e-mail messages appearing to come from legitimate sources like your bank, your Internet Service Provider, eBay, or PayPal, for example. These messages usually direct you to a fake web site and ask you for private information (e.g., password, credit card, or other account updates). The perpetrators then use this private information to commit identity theft.

Warning Sign:

URLs Don't Match - Place your mouse over the link in the e-mail message. If the URL displayed in the window of your browser is not exactly the same as the text of the link provided in the message, run. It's probably a fake. Sometimes the URLs do match and the URL is still a fake.

- Hash Digest changes from

4720eddc b294a991e712b0e930bd5c61d22ac198219c684ed6d6cb
358ac4d564

to

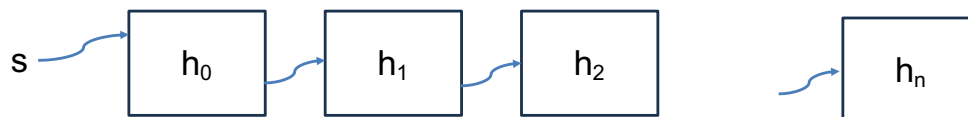
a1125c1a42c1b4335bab10e8aaf1b600a97176ee1e40ae5c390152
8ce0f0bdf1

Hash Chain - Basics

- A Hash chain is a sequence of values where each value is the hash of the previous value.

Example:

$$h_0 = \text{SHA256}(s) : h_1 = \text{SHA256}(h_0) : h_2 = \text{SHA256}(h_1) \quad \dots \quad h_n = \text{SHA256}(h_{n-1})$$



Two key properties

- Knowing h_k , you cannot derive any h_i for $i < k$
- If you change s , you must recompute all hashes

Each value takes the hash of the previous value

One-way property is very useful as we will see in the next example

Hash Chain – OTP Systems

- Client generate K (say $k=1000$) hashes for the password
 - $\text{Hash}^1(\text{password}) = \text{Hash}(\text{password})$
 - $\text{Hash}^n(\text{password}) = \text{Hash}(\text{Hash}^{n-1}(\text{password}))$
- Server also generates $k+1$ hashes,
 - Only stores $\text{H}^{1000}(\text{password})$ (called anchor)
- Client willing to authenticate sends $\text{H}^{1000}(\text{password})$
- Server computes $\text{H}^{1001}(\text{password}) = \text{H}(\text{H}^{1000}(\text{password}))$
 - Match found, store $\text{H}^{1000}(\text{password})$ for next time
- Eavesdropper can't use $\text{H}^{1000}(\text{password})$ since server expects $\text{H}^{999}(\text{password})$
 - You cannot go backwards even if you know the hash function and you have captured $\text{H}^{1000}(\text{password})$

Hash is generated from 1 to 1000.

So H^{1000} is in fact generated using hash of H^{999} .

During the protocol exchange, reverse sequence is used.

So when user logs in, server says send them H^{999} . Now it can run hash of H^{999} and see if it matches stored valued H^{1000} , which was already pre-stored.

The server stores for each user

the user name,

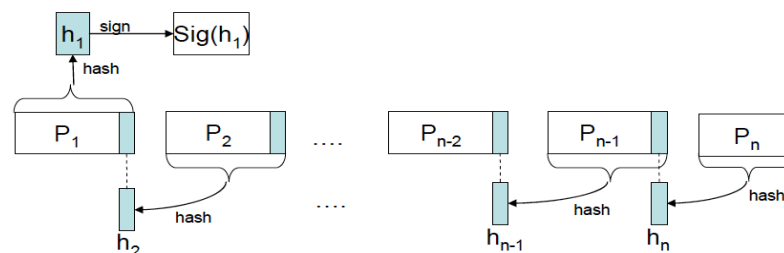
an integer n ,

the n -fold hash of the password: $\text{hash}^n(\text{password})$.

If the user wants to log on, the user types in the password. Her machine sends a request to the server, which answers with a prompt for n . The user's machine calculates $\text{hash}^{n-1}(\text{password})$ and sends this to the server. The server calculates $\text{hash}(\text{hash}^{n-1}(\text{password})) = \text{hash}^n(\text{password})$. If this value matches the one on file, then the login is successful. The server replaces $\text{hash}^n(\text{password})$ with $\text{hash}^{n-1}(\text{password})$ and decrements n . When n reaches 0, the password needs to be reset.

Hash Chain - Code Dissemination

- Fixed size segments – called pages
- Calculate hash value (h_n) of last page P_n and append this to previous page P_{n-1} , until first page is reached
- Hash value of first page P_1 signed with private key (see the next few slides)
- Receivers verify this using public key of sender



If the receiver want to verify all the pages, everything verified sequentially -- starting from the end P_n

Increase the size of data that is being transmitted

If only signed hash is shared it become equal to digital signature for all the packets

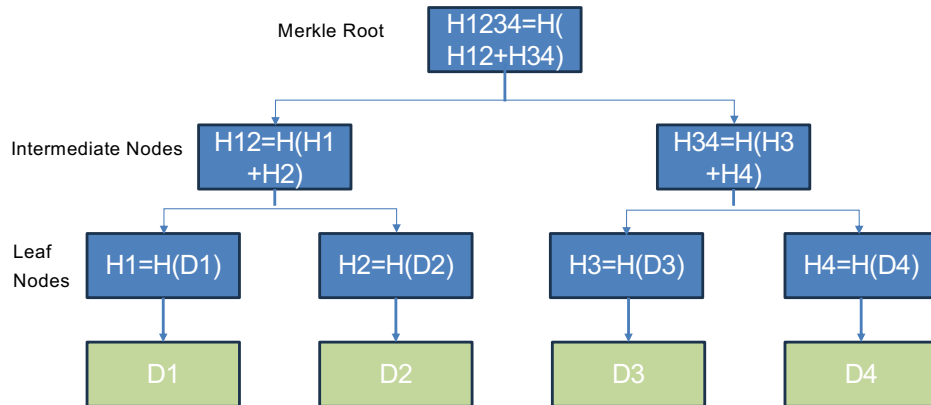
In this case, need access to all pages to formulate – not good for real time streaming etc,

Merkle Hash Tree (MT)

- A binary tree of hashes used to verify the integrity of large sets of data
 - Merkle root is a single hash on the top of the tree that represents the entire dataset
 - Leaf nodes are the hashes of raw data blocks
 - Intermediate nodes are hashes of concatenated child hashes
- Only a small number of hashes are needed to verify a single data block – compare this to Hash Chain

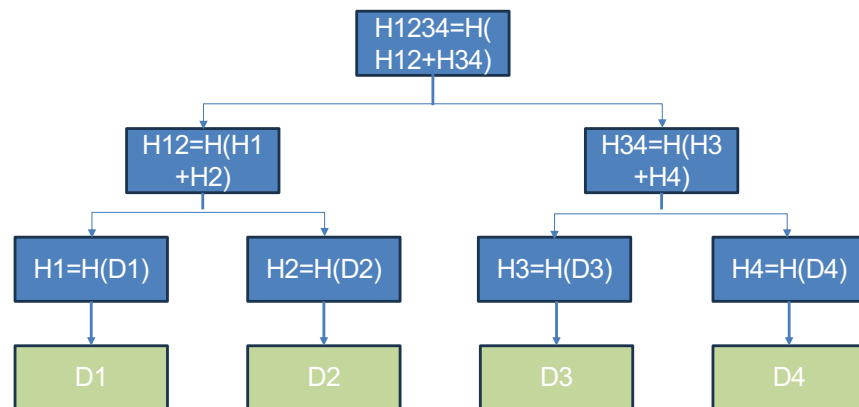
See the next slide

MT Example



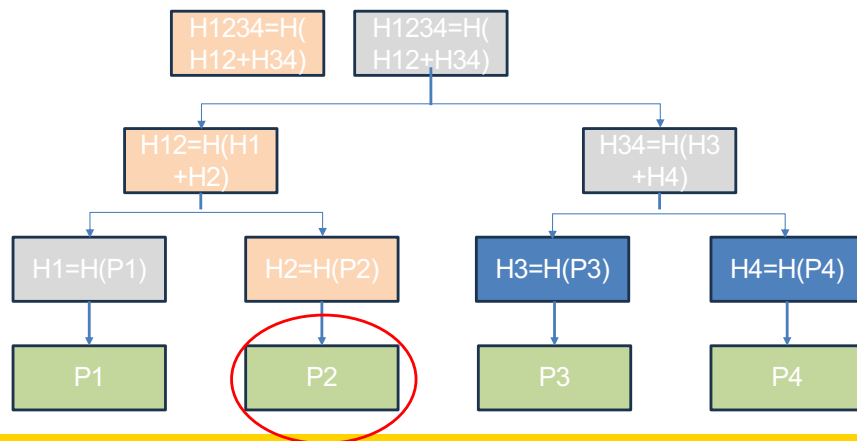
MT – Verify Integrity of Entire Data Set

- Verifier generates the complete MT
- Obtain Merkle Root from a trusted source
- If the MT root matches, entire data is ok.



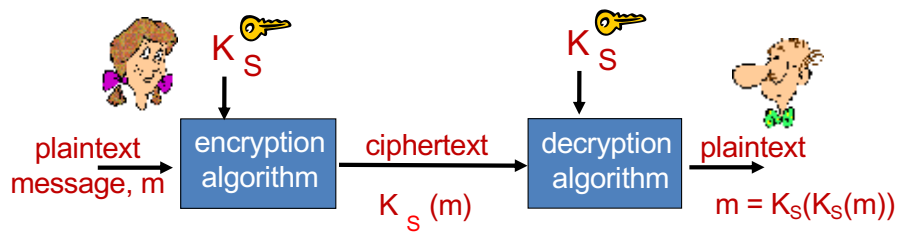
MT – Verify Integrity of a Single Data Block

- Verifier starts from a data element
- Obtain specific Hashes of MT from a trusted source (Merkle proof)
- If the MT root matches, data block is ok.



You only need to know $\log_2(n)$ hashes to verify one block – 10 Hashes for a 1024 block dataset

Symmetric Key Cryptography

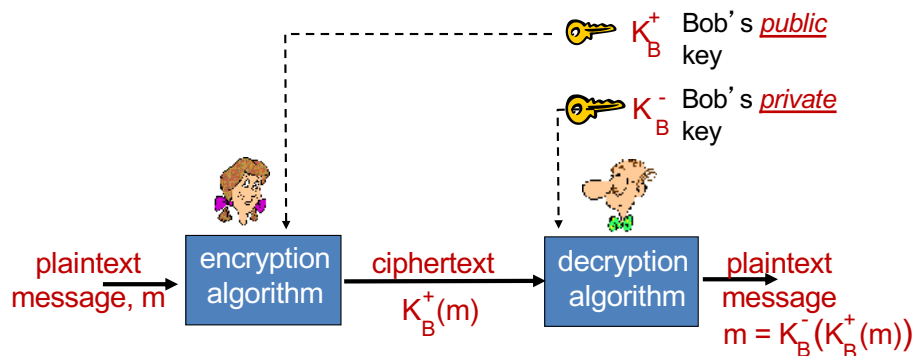


Symmetric key crypto: Bob and Alice share same (symmetric) key: K_S

Q: How do Bob and Alice agree on the key value?

The encryption algorithm is well known, Only the Key K_S is kept secret.
Ciphertext appears as gibberish.
Similar to two persons sharing two duplicate keys of a physical lock.

Public Key Cryptography (PKC)



RSA: Rivest, Shamir, Adelson algorithm

WK 01-02

16



Everyone knows public key of Bob but only Bob knows his private key.

Key pair have been generated such that a message encrypted by Bob's public key (e.g. by Alice) can be decrypted by only Bob using his private key.

Another important property that anything encrypted by private key can be opened by anyone with the public key.

Public-Key Cryptography is the foundation for digital signatures in BCs. It uses a pair of keys to encrypt and decrypt data:

One of the keys is called the *Public key* and is expected to be disseminated widely. The other key is called the *Private key* and is expected to be known only to the owner.

Because these cryptographic systems use a different key, they are also called *asymmetric cryptography*.

It's easy to create new key pairs using algorithms such as RSA and ECC (Elliptic curve cryptography). These algorithms rely on a large number as the seed to generate a key pair, e.g., you can wiggle your mouse several times to generate a large random number.

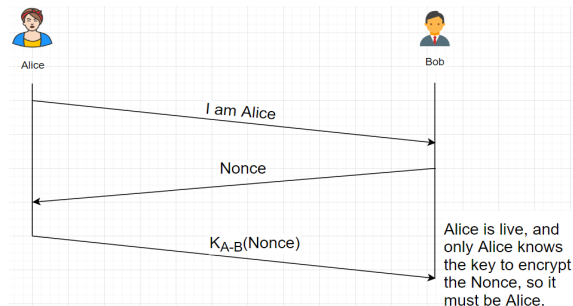
Typical key lengths are 128, 256, 384, 512, 1024, 2048, and 4096 bits.

Longer keys typically enhance security. With RSA we use key lengths over 2048 today. ECC keys are relatively short as they are more effective in withstanding attacks. E.g., 384-bit ECC has similar strength as 4096-bit RSA key.

Bitcoin uses Elliptic Curve Digital Signature Algorithm (ECDSA) with the secp256k1 curve, which generates 256-bit keys.

Authentication: Symmetric keys

- **nonce**: number (R) used only once-in-a-lifetime
 - to avoid playback attacks
- To prove Alice “live”, Bob sends Alice nonce, R. Alice must return R, encrypted with shared secret key



Playback attacks happens at a later time. So Nonce keeps the response fresh.

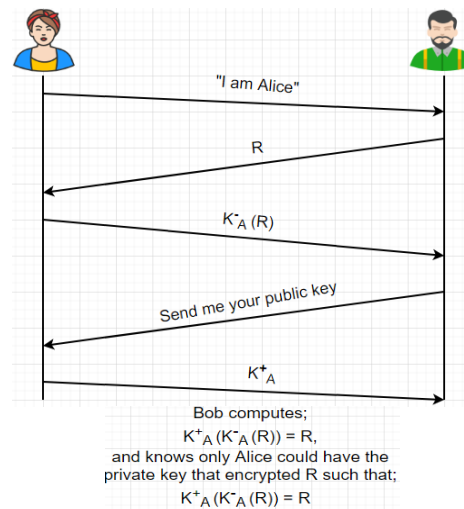
Even if Eve tries the playback attack, Nonce will not be fresh, unless the same Nonce is used later.

Any person in the middle doesn't have the shared key between Alice and Bob making it difficult for a middle attack to happen unless the key is cracked.

Drawback: Needs symmetric key (how to distribute) if same Nonce R used, can be replayed in future by the attacker

Authentication: Asymmetric keys

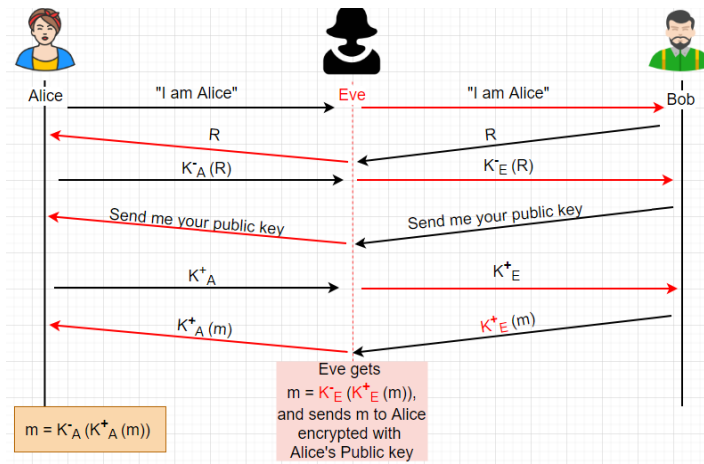
- Can we authenticate using public key techniques?



We can use public key techniques. Alice, receives R , so she can use her private key to encrypt the R (private key means Bob can use Alice's public key) but Bob needs to find out the public key, so what Bob does is to request for Alice's public key. Alice gives her public key to Bob, then Bob computes the mathematical operation to find R and compare it with the R sent earlier by Bob to Alice.

A Security Hole?

- **Person in the middle attack:** Eve poses as Alice (to Bob) and as Bob (to Alice)



Both Eve and Alice receive m .

What is the problem with this scheme?

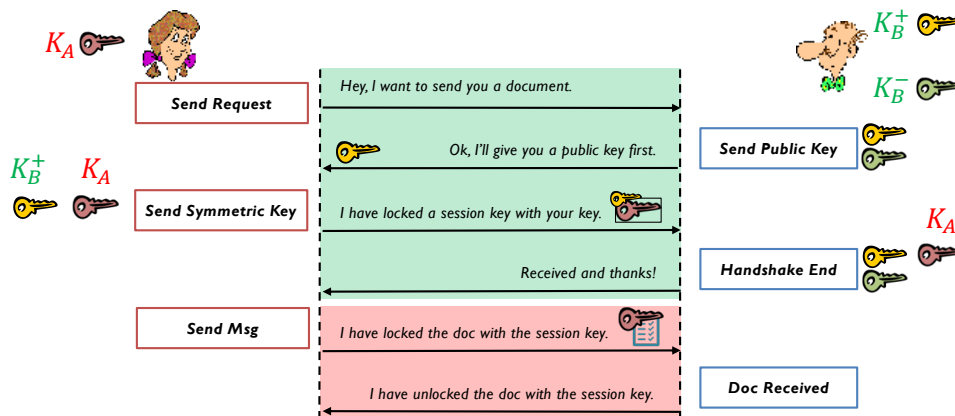
The weakness is Bob is asking Alice to provide the key but Bob could not verify if the public key sent belongs to Alice.

Need to use PKI (Public Key Infrastructure).

Encryption in Practice : A hybrid approach

Alice wants to send Bob a document

- Exchange session (symmetric) key with *asymmetric* encryption
- Send msg with *symmetric* encryption



To build a secure connection, Bob will send his public key, K_B^+ , to Alice.

Then, Alice will encrypt a session key, K_A , with bob's public key, so that nobody can decrypt the session key except Bob. And then send it to Bob.

So, Alice can now directly encrypt the file with the session key and send it to Bob.

As Bob has already gotten the shared session key from Alice, he can decrypt the received file.

We can see that symmetric encryption secured the transmission of document.

Once Alice receives Bobs public key, it can use that to encrypt the whole document?

It is possible but expensive to use Asymmetric keys for encrypting large messages.

Also, Bob's public key is known to all while the session key is only known to 2 persons – Bob and Alice.

Encryption in Practice #1

Scenario A

1. Mallroy intercepts K_A encrypted with K_B^+ from Alice.
2. Mallroy sends Bob K_M encrypted with K_B^+ .
3. Mallroy intercepts the document encrypted with K_A .
4. Mallroy sends Bob a changed document encrypted with K_M .

Question

- Can Bob figure out if the document sent to him has changed?

But is it really secure?

😊 Let's consider a scenario that could happen in practice

Mallroy, intercepts both the encrypted document and encrypted shared symmetric key from Alice.

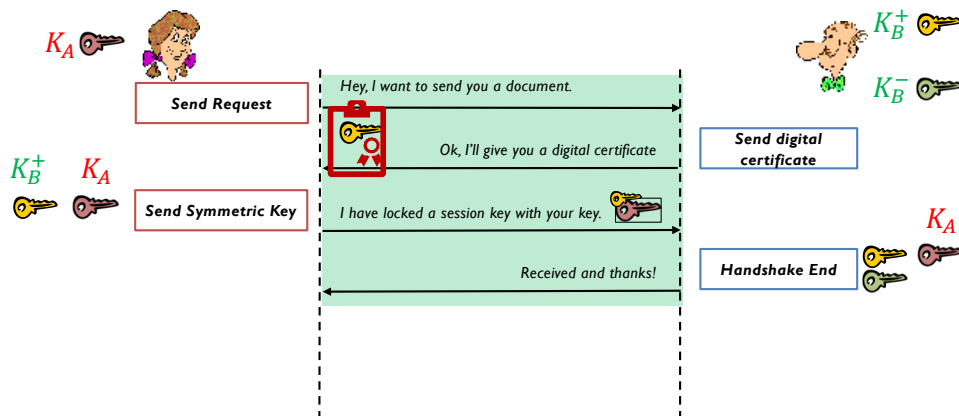
And then, she pretends to be Alice, and sends her own session key and a changed document encrypted with her key, K_M .

😊 Now, think about it, can Bob figure out whether he received the correct file?

How can we solve this problem?

RSA-Based Key Exchange

- Involve PKI
- Exchange session (symmetric) key with *asymmetric* encryption



A digital certificate establishes the ownership of a public key with a specific entity.

When Alice received the digital certificate, it can verify its authenticity by contacting a Validation Authority, that is part of the PKI.

Digital signatures

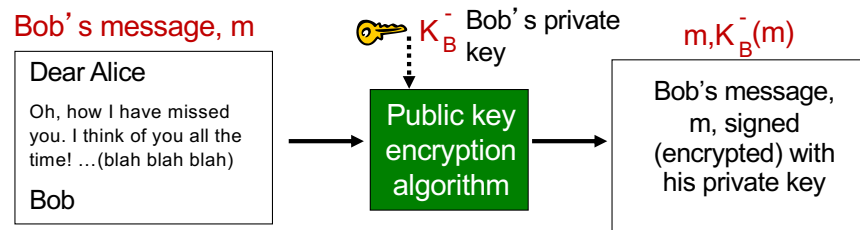
Cryptographic technique analogous to hand-written signatures:

- Sender (Bob) digitally signs document, establishing he is document owner/creator.
- **Verifiable, non-forgeable:** recipient (Alice) can prove to someone that Bob, and no one else (including Alice), must have signed document

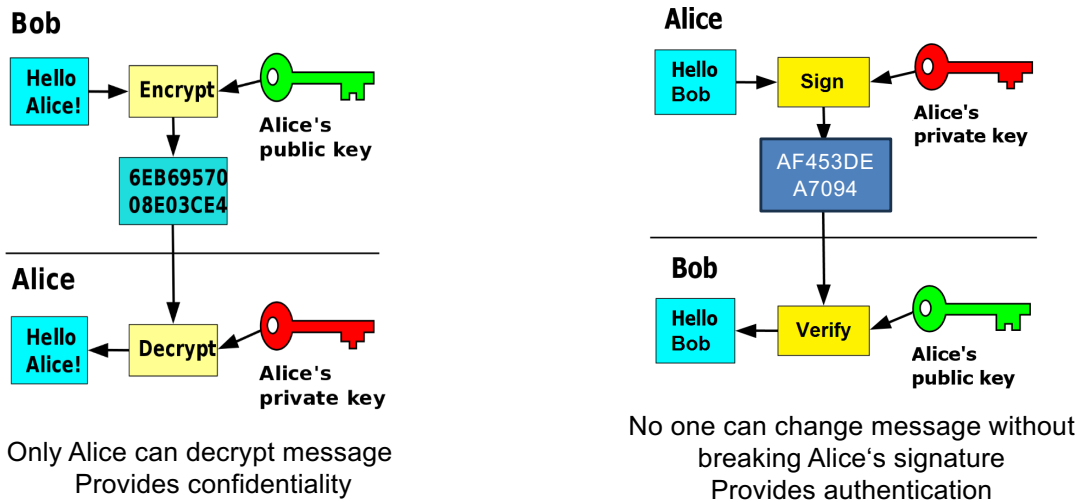
Digital signatures

Simple digital signature for message m :

- Bob signs m by encrypting with his private key K_B , creating "signed" message, $K_B^-(m)$



Encryption vs Digital Signatures



WK 01-02

25



- Public-Key Cryptography serves 2 purposes, where it can be used for encryption and digital signatures
- Encryption
 - On the left, Bob wants to send a secret message to Alice.
 - So, he encrypts the message using Alice's public key, which is well known. Anyone who knows Alice's public key can send her a secure message.
 - However, to decrypt the message Alice needs to use her private key. As Alice is the only one who knows the private key, no one can read the message other than Alice.
 - So, use the public key to encrypt and the private key to decrypt.
- Digital signatures
 - In digital signatures, the use of key pairs is reversed.
 - In this case, Alice wants to prove that she is the one who signed the

message, i.e., Alice is trying to prove her authenticity (much like putting her signature on a paper).

- So, the message is signed with Alice's private key and Bob or anyone else can verify it using her public key.
- No one else can impersonate Alice unless they know her private key
- In practice, what is signed is a datum/hash of the message, which is not shown in the diagram to keep it simple. As a hash is much smaller than a document or TX, it is faster to sign on a hash than the original data.

Digital signatures

- Suppose Alice receives msg m , with signature: $m, K_B^-(m)$ -
- Alice verifies m signed by Bob by applying Bob's public key K_B^+ to $K_B^-(m)$ then checks $K_B^+(K_B^-(m)) = m$.
- If $K_B^+(K_B^-(m)) = m$, whoever signed m must have used Bob's private key.

Alice thus verifies that:

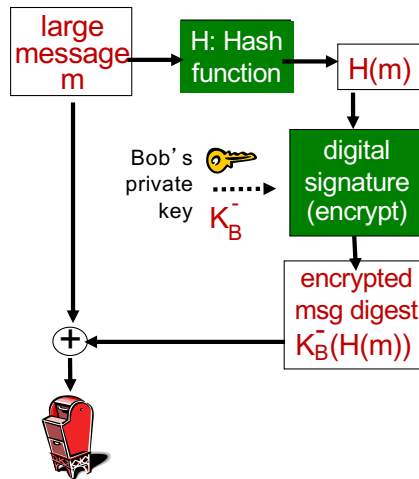
- Bob signed m
- no one else signed m
- Bob signed m and not m'

non-repudiation:

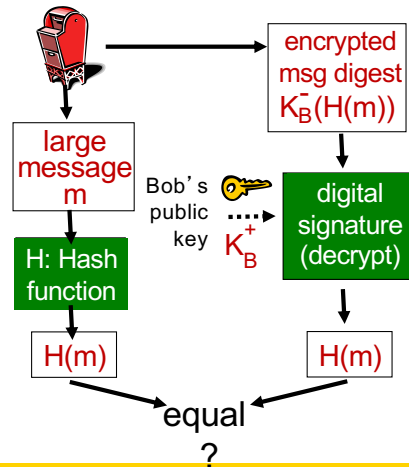
- ✓ Alice can take m , and signature $K_B^-(m)$ to court and prove that Bob signed m

Digital signature = signed message digest

Bob sends digitally signed message:



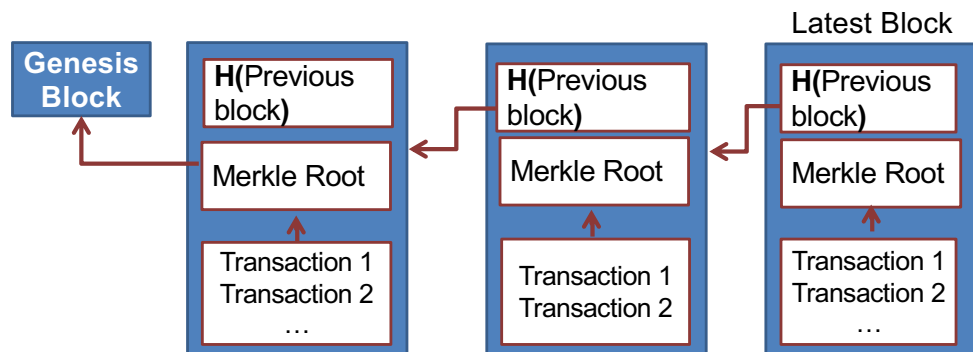
Alice verifies signature, integrity of digitally signed message:



Use of Cryptography in Blockchain

Blockchain is a replicated and distributed ledger

- Merkle Trees (MT) and Hash Chains
 - Ordered Transactions in a block in form of a Merkle Tree
 - Store the root of the MT in the block's header
 - Create a Hash Chain linking the block to the previous block



BCs goal is to replace the central trusted authority with a network of computers such that we establish a decentralised, trustless environment.

From an implementation point of view, BC is a replicated or distributed ledger which looks like a linked list with hash pointers distributed over a network of nodes.

A Merkle Tree is a way of efficiently & securely verifying the contents of a large collection of data. In BCs, it is used to summarize the set of TXs included in a block. So even if a single TX changes, the root hash will change.

Also, in BCs, we can verify whether a given TX is included in a block using the Merkle Tree. Given a TX and hashes of its adjacent and parent hashes, we can verify whether the same root hash is produced in $\Theta(\log n)$ time. For example, the block header will contain only the root hash, To check whether TX_4 is in the block, we need only to compute $H(TX_4)$, concatenate it with $H(TX_3)$ and find the next level hash. Then concatenate that hash with the hash from the left of the tree (i.e., $H(H(TX_1) + H(TX_2))$) and calculate the next level hash, and so on..,

Such a proof is called a *Merkle proof*.

In Bitcoin, if there are an odd number of TXs, the TX without a partner is hashed with a copy of itself. Similarly, any hash without a partner is hashed with itself.

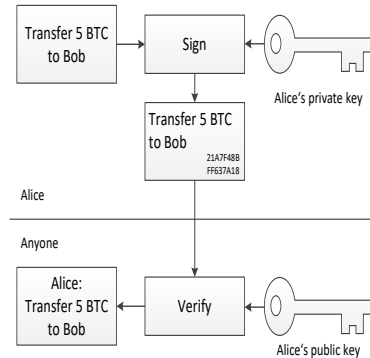
Use of Cryptography in Blockchain

Use private key to control an account

- Control means the ability to act on behalf of the account
- E.g., spending assets it owns

Each account is known by its public key

- "Alice" here is really
0x7a2f16dab8b5c2cf99c35e4c6a5beb45c7df8f87 → Pseudo-anonymous
- For some accounts, we may know the person/organisation owning it
- But by default, we don't



Here we talk about the specific use of public-key cryptography in BCs.

To transfer 5 Bitcoin (currency symbol BTC) to Bob, Alice needs to prove to Bitcoin miners that she controls the transfer account.

Ownership of an account in BC is proven using the private key, as only the owner of the key is supposed to know it.

Therefore, Alice signs the TX using her private key.

Whereas each account is known based on its public key. Hence, anyone can use her public key to verify that the TX was signed by Alice. In practice, an account in Ethereum is derived from a 256-bit public key and reduced to a 160-bit address for convenience. The private key is still 256-bits.

Note that Alice's account is represented as a bit string like in the slide.

Such a bit string gives pseudo-anonymity than keeping track of Alice's name (more on this later).

However, for some accounts, we may know the person/organization owning it. But by default, we don't know.

In fact, in Bitcoin, it's even recommended to create a new account for each new TX such that you can achieve some level of anonymity. In BCs, public-key cryptography is used to indicate the ownership of assets and authentication to spend them. E.g., when Alice sends \$300 to Bob, she needs to sign her TX using her private key (as she's the only one that is supposed to know the private key). If Alice loses her private key, it's the same as her losing control of her assets.

Summary

Lot of these crypto mechanism are building blocks for security protocols

- Acknowledgement:
 - Adaptation of foils from Kurose/Ross (revision from basic networking subject COMP3331)
 - Stallings: Network Security Essentials, Chapter 2 and 3 has good summary of Symmetric Encryption, Hash etc.
 - [Cryptography KA](https://www.cybok.org/) from <https://www.cybok.org/>